SOFTWARE ARCHITECTURE AND DESIGN SPECIFICATION

SimpleShell — Command-Line Interpreter

Project #18 | Version 2.1 | C11 | Linux / POSIX

Project Team

Name	SRN
Rishab Naveen	PES1UG24CS710
Aadarsh Reddy Yedhala	PES1UG24CS706
Vatsal Gupta	PES1UG24CS702
Vineet	PES1UG24CS698

1. Introduction

This document describes the logical architecture and design of SimpleShell, a Unix-like command-line interpreter in C. It complements the SRS and Test Plan. The architecture below describes responsibilities and interfaces; filenames are illustrative logical modules and must be reconciled with the source tree before being treated as implemented files.

2. System overview

SimpleShell reads a command line from the terminal, tokenises and parses it, expands supported variables, then dispatches built-in commands or launches an external program. POSIX process and file-descriptor operations support child execution, redirection, and pipelines. The shell returns to the prompt until the user exits or reaches EOF.

3. Layered architecture

USER / TERMINAL
REPL: prompt → read line → dispatch → repeat
LEXER / PARSER: tokens → command representation
EXPANSION: environment values and status/PID forms
EXECUTOR / BUILT-INS
EXTERNAL EXECUTION: PATH lookup → fork → execvp → waitpid
I/O REDIRECTION + PIPELINE FILE-DESCRIPTOR SETUP
POSIX KERNEL: processes, pipes, signals, files

4. Module responsibilities

Logical module	Responsibility
Entry point / shell state	Initialises shell state and signal policy; starts the REPL.
REPL	Displays prompt, reads lines, handles EOF and dispatches commands.
Lexer	Recognises words, operators, quoting and token boundaries.
Parser	Builds command/pipeline representation and validates redirection syntax.

Logical module	Responsibility
Variable expansion	Expands only the variable/status/PID forms defined by the SRS.
Built-ins	Implements cd, pwd, echo, export, unset, help and exit.
External executor	Searches PATH and starts external programs with fork() and execvp(); waits with waitpid() where applicable.
I/O redirection	Opens requested files and uses dup2() to connect standard descriptors.
Pipeline manager	Creates pipes and connects pipeline stages; closes unused descriptors and waits for children.
Signal handling	Defines parent/child signal behaviour in accordance with the SRS.
Environment and utilities	Wraps environment operations and shared string/error/memory helpers.

5. Data design

The implementation may use token and command structures similar to the following. These are design-level examples; the actual field names and memory ownership must match the source code.

token_t: token type plus token text/value, used by the lexer and parser.

cmd_t: NULL-terminated argv, optional input/output/append/stderr filenames, and a link to the next pipeline stage.

shell state: last command status, shell PID, exit-request flag, and exit code. Prefer explicit state passed to functions over undocumented global state.

6. Execution flow

1. Read a line from the terminal and handle EOF/blank input.
2. Lex the line into tokens; report malformed quoting or operators safely.
3. Expand the variable forms specified by the SRS.
4. Parse tokens into a command or pipeline representation.
5. If the command is a built-in, invoke the built-in handler in the appropriate process context.
6. Otherwise, resolve the external command using PATH and execute it using fork() and execvp().
7. Configure redirections and pipe endpoints with open(), pipe(), dup2(), and close() as required.
8. Collect child status with waitpid() and update the shell's last-status value.
9. Release allocated tokens/command structures and return to the REPL.

7. Design decisions and constraints

Decision	Rationale
Use POSIX process APIs	Direct process creation and exec avoids delegating the whole command to system() and matches the documented shell design.
Use execvp for PATH lookup	execvp searches PATH; use this consistently in the architecture and test plan. If the implementation uses execve with a custom resolver, update both documents to describe that exact approach instead.
Represent pipeline stages explicitly	A command chain/list or equivalent structure supports a variable number of stages.
Keep built-ins separate	Built-ins such as cd must affect the shell process when their semantics require it.
No GNU Readline in v1.0	Avoids an extra library dependency; command history and tab completion remain out of scope.
Parent/child signal policy	The shell should survive interactive interrupts; foreground child behaviour must match the SRS and be tested manually.

8. Non-functional considerations

Memory safety: free dynamically allocated tokens and command structures on every path, including parse and execution errors. Input safety: enforce the SRS input limit and check allocation/string operations. Portability: target C11 and POSIX APIs on supported Linux/POSIX environments. Build quality: compile with the warning flags required by the project. Security: follow the SRS PATH-search policy and avoid executing a local file implicitly unless the policy permits it.

9. Consistency with the SRS and Test Plan

The architecture uses the same functional groups FR-01 through FR-08 as the Test Plan: REPL, parsing, built-ins, external execution, redirection, pipelines, signals, and variable expansion. Non-functional concerns map to the test plan's NFR checks for performance, memory safety, long-input robustness, portability, and PATH handling. Acceptance-criteria wording and exact requirement IDs must be checked against the approved SRS before submission.

Implementation caveat: the uploaded item labelled as the SRS DOCX was actually a GitHub HTML page, not the underlying Word document. These revised documents therefore align with the requirement groups and feature scope visible in the existing Test Plan and Architecture PDF; they cannot be certified as an exact line-by-line match to the SRS until the real DOCX is provided.